Docxtract FYP - Local Mistral Model Setup

---

1. Objective

Replace cloud-based Gemini 2.5 Flash API with a local, offline model for Docxtract to handle: - PDF summarization - Chat with PDF - Insight generation

Hardware: Intel i5 8th Gen, 16 GB RAM

---

2. Model Selection

Primary Model (Manual Download)

- **Mistral-7B-Instruct-v0.2.Q4_K_M.gguf**
- CPU-friendly (~6-7 GB RAM)
- Instruction-tuned (good for summarization, Q&A, insights)
- Quantized GGUF format for offline inference

Auxiliary Models (Auto-Downloaded via Python)

- **Embeddings**: sentence-transformers/all-MiniLM-L6-v2
- **OCR**: PaddleOCR models (automatic download)
- **PDF extraction**: pdfplumber (no model needed)
- **Vector store**: FAISS (no model needed)

---

3. Backend Stack

- **FastAPI**: For frontend-backend communication
- **llama-cpp-python**: For running Mistral locally
- **PaddleOCR / pdfplumber**: For text extraction
- **FAISS**: For vector search
- **MiniLM**: For embeddings

---

4. Installation Commands

```
pip install fastapi uvicorn llama-cpp-python
pip install paddleocr pdfplumber
pip install sentence-transformers faiss-cpu
```

---

5. Model Loading (Python)

```python
from llama_cpp import Llama

llm = Llama(
    model_path="models/mistral-7b-instruct-v0.2.Q4_K_M.gguf",
    n_ctx=4096,
    n_threads=8,
    verbose=False
)
```

---

6. PDF Text Extraction

Digital PDFs

```python
import pdfplumber

def extract_text(pdf_path):
    text = ""
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text += page.extract_text() or ""
    return text
```

Scanned PDFs

```python
from paddleocr import PaddleOCR
ocr = PaddleOCR(use_angle_cls=True, lang="en")

def ocr_pdf(images):
    text = ""
```

```
    for img in images:
        result = ocr.ocr(img)
        for line in result[0]:
            text += line[1][0] + " "
    return text
```

---

7. Text Chunking

```
def chunk_text(text, chunk_size=600, overlap=100):
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start+chunk_size])
        start += chunk_size - overlap
    return chunks
```

---

8. Embeddings & FAISS Index

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

def build_index(chunks):
    embeddings = embedder.encode(chunks)
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(np.array(embeddings))
    return index, embeddings
```

9. Retrieval

```
def retrieve(query, chunks, index):
    q_emb = embedder.encode([query])
    _, ids = index.search(np.array(q_emb), k=4)
    return "\n".join([chunks[i] for i in ids[0]])
```

10. Query Mistral

```
def ask_mistral(context, question):
    prompt = f"""
    You are an intelligent document assistant.

    Context:
    {context}

    Question:
    {question}
    """

    output = llm(prompt, max_tokens=400)
    return output["choices"][0]["text"]
```

11. FastAPI Endpoint

```
from fastapi import FastAPI, UploadFile

app = FastAPI()

@app.post("/chat-pdf")
async def chat_pdf(file: UploadFile, question: str):
    path = f"temp/{file.filename}"
    with open(path, "wb") as f:
        f.write(await file.read())

    text = extract_text(path)
    chunks = chunk_text(text)
    index, _ = build_index(chunks)
    context = retrieve(question, chunks, index)

    answer = ask_mistral(context, question)
    return {"answer": answer}
```

---

12. Notes & Tips

- Use `n_threads = number of CPU cores` for speed
- Cache embeddings per PDF
- Chunk PDFs for large documents

- Keep context ≤ 4096 tokens

13. FYP Statement

> “The system replaces cloud-based LLM APIs with a locally hosted quantized Mistral-7B-Instruct model. A Retrieval-Augmented Generation pipeline ensures efficient and accurate document-aware responses while preserving privacy and offline functionality.”

---

14. Minimal Required Files

```
/models
 └── mistral-7b-instruct-v0.2.Q4_K_M.gguf    ✅ (manual download)
```